

HelixSpecifier

HelixSpecifier

Разработка на основе спецификаций с автоматическим масштабированием ритуалов под объём работы.

Краткое описание

HelixSpecifier — это движок Go, объединяющий три методологии разработки: рабочий процесс на основе спецификаций SpecKit, дисциплину TDD от Superpowers и жизненный цикл vех GSD — в единый адаптивный поток. Он классифицирует задачи по уровню усилий и динамически регулирует объём процессов в зависимости от этого.

Описание

HelixSpecifier — это движок для разработки на основе спецификаций (SDD), написанный на Go (модуль `digital.vasic.helixspecifier`) и являющийся частью ансамбля HelixAgent для AI-агентов. Он объединяет три практики разработки, обычно существующие в виде отдельных инструментов и требующие разных подходов, в единый адаптивный рабочий процесс: семифазный процесс SDD от SpecKit (Constitution, Specify, Clarify, Plan, Tasks, Analyze, Implement), тест-ориентированную дисциплину Superpowers с параллельным выполнением подзадач и управление жизненным циклом вех от GSD. Каждый из трёх столпов продолжает выполнять свою функцию, но движок обеспечивает их слаженную работу как единого потока, а не как сшитого вручную конвейера.

Ключевая идея — *адаптивный ритуал*: движок классифицирует входящие задачи по уровню усилий и масштабирует объём процессов под них, чтобы однострочное исправление не проходило через тот же громоздкий ритуал, что и крупная функция, а крупная функция не получала поверхностную проверку, как опечатка. На этой основе

реализованы десять ключевых возможностей: параллельное выполнение задач с ограничением конкуренции, машиночитаемый «Constitution как код», автоматически применяющий обязательные правила, «Найквист-TDD», отслеживающий и обеспечивающий минимальное соотношение тестов к коду, многократные многоагентные дебаты для уточнения спецификаций, адаптивное обучение навыкам, анализ унаследованного кода, предсказательное формирование спецификаций на основе исторических данных, перенос знаний между проектами, динамическая подстройка ритуалов в процессе выполнения и постоянная память спецификаций с семантическим поиском.

Движок подключается как модуль Go — через `go get` или локальную директиву `replace` — и работает за компактным интерфейсом API: достаточно зарегистрировать три столпа, указать масштабировщик ритуалов и память спецификаций, классифицировать объём работы, запустить полный цикл и получить результат с оценкой качества. Поверхность проста, но оркестровка за ней — нет. Как и остальные инструменты семейства Helix, он разрабатывается в рамках режима антиблефа с проверкой на реальном коде, а не на моках, с использованием встроенного раннера вызовов.

Зачем мы его создали

Разработка на основе спецификаций, строгий TDD и управление вехами — это обычно три отдельные практики с тремя разными инструментами. HelixSpecifier был создан, чтобы AI-агент (HelixAgent) мог выполнять все три как единый согласованный и самомасштабируемый рабочий процесс, а не сшивать их вручную.

Почему это меняет правила игры

Система автоматически приводит процесс в соответствие с объёмом работы. Обычно команды оказываются в одной из двух неудачных крайностей: либо избыточная формализация всего подряд (надёжно, но медленно и вызывает скрытое недовольство), либо полное её отсутствие (быстро, пока не становится слишком поздно). HelixSpecifier устраняет этот компромисс, подстраивая уровень формальностей под классифицированную трудоёмкость каждой задачи и корректируя его в реальном времени по мере раскрытия сути работы. Раньше это было невозможно: процесс, который сам определяет свой масштаб для каждой задачи, а вдобавок решения по спецификациям подкрепляются многораундовыми

дебатами с участием нескольких агентов и оценкой позиций — вместо того, чтобы полагаться на первое предположение одного разработчика.

Что нового

- **Адаптивная формализация** — уровень процесса определяется на основе метрик качества в реальном времени и корректируется динамически, а не фиксируется заранее.
- **Найквист-TDD** — контроль соотношения тестов к реализации (минимум 2:1), основанный на логике теоремы Найквиста о дискретизации: чтобы точно зафиксировать поведение, частота выборки должна значительно превышать его скорость, поэтому тесты должны опережать код, который они покрывают.
- **Архитектура дебатов** — многораундовое уточнение спецификаций с участием нескольких агентов, где позиции выдвигаются, оцениваются и согласовываются, заменяя единоличное мнение состязательным процессом.
- **Прогнозирование спецификаций и перенос знаний между проектами** — движок анализирует накопленные потоки данных, чтобы предвосхищать требования и переносить ценные наработки из одного проекта в другой.
- **Constitution как код** — обязательные правила проекта переводятся в машиночитаемый формат и обеспечиваются исполнением движка, а не бдительностью ревьюеров.

Основные технические вызовы и их решения

- **Объединение трёх методологий без конфликтов** — SpecKit, Superpowers и GSD предполагают, что каждая из них контролирует рабочий процесс. Решение: интеграционный движок, который регистрирует каждый компонент за общим интерфейсом и управляет ими через единый жизненный цикл потока, превращая три разрозненных процесса в один слаженный.
- **Определение необходимого уровня формализации для конкретной задачи** — завышенная оценка замедляет всё, заниженная позволяет рискованной работе уйти в продакшен без проверки. Решение: классификатор трудоёмкости, который оценивает объём работы и передаёт данные в скейлер формализации, динамически корректирующий уровень процесса по мере выполнения.
- **Поддержание высокого качества спецификаций без человеческого контроля на каждом этапе** — решение: замена одноразовых спецификаций процессом уточнения

на основе дебатов, где агенты оценивают конкурирующие позиции в несколько раундов, а также соблюдение соотношения Найквист-TDD, не позволяющее реализации опережать тесты.

Технический стек

- **Go** — выбран, чтобы движок поставлялся в виде единого импортируемого бинарного файла без зависимостей; его модель конкурентности делает возможной параллельную диспетчеризацию задач с ограниченным числом потоков и многораундовые дебаты между агентами, избавляя от головной боли с многопоточностью.
- **logrus** — структурированное логирование, пронизывающее движок и все три компонента, благодаря чему решения потока (классификация, изменения формализации, результаты дебатов) остаются читаемыми постфактум.
- **Компонент SpecKit** — семифазный процесс разработки на основе спецификаций (Constitution → Спецификация → Уточнение → Планирование → Задачи → Анализ → Реализация), обеспечивающий дисциплинированную основу для превращения спецификации в код.
- **Компонент Superpowers** — дисциплина TDD с параллельным исполнением подзадач агентами, гарантирующая строгость тестирования и распараллеливание, которое поддерживает честность и скорость реализации.
- **Компонент GSD** — управление вехами и жизненным циклом, придающее потоку понимание «завершённости» и последовательности этапов.
- **Хранилище памяти спецификаций** — постоянный, семантически поисковый индекс прошлых спецификаций, основа для прогнозирования требований и переноса знаний между проектами, исключая необходимость начинать с нуля каждый раз.

Заметки о статусе и добросовестности

- **Статус: бета-версия.** Используется в качестве компонента модуля Go в составе HelixAgent.
- **Лицензия: не определена.** Лицензия не была обнаружена через GitHub API — НЕПРОВЕРЕНО / не заявлено.
- Отображаемое имя “HelixSpecifier” соответствует репозиторию specifier.

Приоритетный уровень: Helix-первичный.